Recherche de la section du C-foil par co-krigeage séquentiel ciblé par la VPP

Description exacte de l'algorithme et de l'architecture

`Multifidelity-GP-Clean/vpp_acceleration` · branche `feat/vpp-acceleration` · état du code du 3e vrai run (`runs/2026-10-22_1519_real_npllt_xfoil_fsi`) · 22/10/2026

Table des matières

1 Ce que fait l'algorithme, en une page **1**

2 Le problème d'optimisation **3**

3 La VPP : MDS, Intens SY, variante `CFoilFSI` **3**

3.1 Le système . . . 3

3.2 Le problème résolu à chaque condition . . . 3

3.3 Le foil dans la VPP . . . 4

3.4 La correction FSI (foil flexible) . . . 4

3.5 Les sensibilités de vitesse . . . 5

4 Les codes du torseur **5**

5 La section : Kulfan, famille admissible, PCA **6**

6 Le surrogate du torseur : 7 co-krigeages AR(1) **7**

7 Le critère de raffinement ciblé par la VPP **7**

8 Les lots en parallèle : le « kriging believer » **8**

9 L'optimiseur de section **8**

10 Le plan de départ **9**

11 L'architecture logicielle et parallèle **10**

12 La validation finale **10**

13 L'analyse sur une distribution d'usage **11**

14 Les réglages du 3e run **12**

15 Limites connues et évolutions prévues **12**

1 Ce que fait l'algorithme, en une page

Le but. On cherche la section (le profil) du C-foil de l'Intens SY qui maximise la vitesse du bateau donnée par la VPP de MDS, moyennée sur cinq conditions de vent. La forme en plan du foil est fixée, et seule la section change.

La difficulté. La VPP appelle le modèle du foil des centaines de fois par point de polaire. Les codes aérodynamiques (NPLLT, AVL avec XFOIL) sont trop lents pour cela. On remplace donc le foil, dans la VPP, par un **surrogate** de son torseur : un co-krigeage multi-fidélité, dont les entrées sont la section et l'attitude du foil. Ce surrogate s'améliore pendant la recherche. On ajoute des calculs de code **là où la VPP travaille** (les états d'équilibre des sections candidates) et **sur ce qui compte pour la vitesse** (chaque coefficient est pondéré par $(\partial V/\partial C_k)^2$).

La boucle. Elle alterne deux étapes jusqu'à une limite de temps.

1. **La mise à jour de l'optimiseur.**

— On crible 256 sections nouvelles au premier ordre, avec le surrogate et les sensibilités de vitesse de la meilleure section du moment (le « titulaire »).
— On résout la VPP complète sur les 3 meilleures et sur le titulaire, puis le titulaire devient la meilleure des quatre.
— Les états d'équilibre de ces 4 sections, aux 5 conditions, donnent les points d'intégration du critère de raffinement.

2. **Un lot de raffinement.** On choisit 16 couples (point, niveau de code), l'un après l'autre, par la meilleure réduction de variance de la vitesse par unité de coût. Puis les codes du lot tournent en parallèle, et leurs résultats entrent dans le co-krigeage.

Les niveaux de code (3e run).

— **Niveau 1 (BF) :** ligne portante non plane NPLLT avec les polaires NeuralFoil xxxlarge.
— **Niveau 2 (HF) :** la même ligne portante, avec les polaires XFOIL.
— **Niveau 3 :** AVL × XFOIL, la chaîne de la RS de production. Il ne sert qu'à la validation finale.

Le foil est flexible. Le surrogate et les codes sont rigides. La VPP ajoute une correction de déformée, tirée de la RS FSI de production et mise à l'échelle de la raideur de la section (§ 3.4).

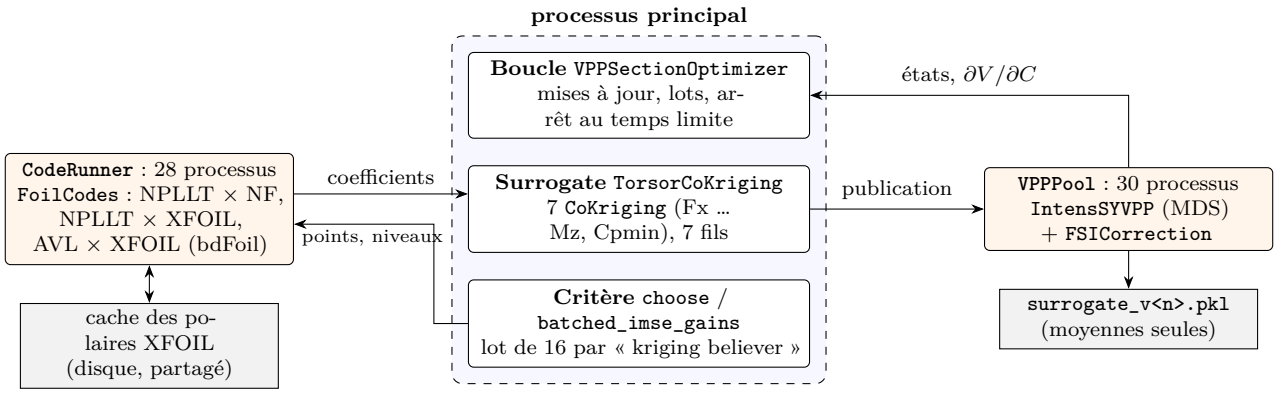


FIGURE 1 – Les trois groupes de processus d'un run (§ 11). Les deux pools ne travaillent jamais en même temps : la boucle est synchrone.

À retenir

— **Ce qui est optimisé :** 10 scores PCA d'une section de Kulfan (16 poids), dans une famille de sections admissibles.
— **Ce que voit la VPP :** la moyenne du co-krigeage, plus la correction FSI.
— **Ce qui est raffiné :** le co-krigeage, autour des états VPP des sections en compétition.
— **Ce qui décide :** la vitesse moyenne sur 5 conditions, calculée par la VPP complète sur 4 sections à chaque mise à jour.
— **Ce qui s'analyse ensuite :** le comportement des sections retenues sur une distribution d'usage (TWA, TWS), donnée à chaque run (§ 13). Elle ne change pas la recherche.

Notations

$\boldsymbol{z} \in \mathbb{R}^k$, $k = 10$	scores PCA de la section (blanchis, de variance 1 sur la famille)
$\boldsymbol{w} = \mathcal{D}(\boldsymbol{z}) \in \mathbb{R}^{16}$	poids de Kulfan décodés (8 extrados, 8 intrados)
$\boldsymbol{a} = (a_c, a_r, a_y)$	attitude du foil : `Cant_Eff`, `Rake_Eff`, `Yaw_Eff` [°]
$\boldsymbol{x} = (\boldsymbol{z}, \boldsymbol{a}) \in \mathbb{R}^{13}$	entrée du surrogate
$C_k(\boldsymbol{x})$, $k = 1..7$	sorties : $F_x, F_y, F_z, M_x, M_y, M_z$ (coefficients, moments en m) et $C_{p,\min}$
f_l, $l = 1, 2, 3$	codes : NPLLT × NeuralFoil, NPLLT × XFOIL, AVL × XFOIL
$\mu_k(\boldsymbol{x})$, $s_k^2(\boldsymbol{x})$	moyenne et variance a posteriori du co-krigeage de la sortie k (niveau 2)
$c = (\mathrm{TWA}_c, \mathrm{TWS}_c)$	condition de navigation, $c = 1..5$
$\boldsymbol{u} \in \mathbb{R}^7$	inconnues de la VPP : vitesse, dérive, gîte, assiette, déplacement de carène, safran, réduction de voilure (`Flat`)
$V^\star(\boldsymbol{z}; c)$	vitesse d'équilibre maximale de la section $\boldsymbol{z}$ à la condition c
$J(\boldsymbol{z})$	objectif : $\frac{1}{5}\sum_c V^\star(\boldsymbol{z}; c)$
d_{kc}	sensibilité $\partial V^\star / \partial C_k$ à l'état du titulaire, condition c [nd par unité]

2 Le problème d'optimisation

$$\max_{\boldsymbol{z}} \; J(\boldsymbol{z}) = \frac{1}{5}\sum_{c=1}^{5} V^{\star}(\boldsymbol{z}; c) \qquad \text{sous la contrainte} \qquad \mathcal{D}(\boldsymbol{z}) \in \mathcal{F},$$

où $\mathcal{F}$ est la famille des sections admissibles (§ 5).

— **Les conditions.** Ce sont les couples (TWA, TWS) : (45°, 12 nd), (45°, 16 nd), (60°, 16 nd), (75°, 16 nd) et (90°, 14 nd). Elles ont le même poids.

— **La vitesse** $V^{\star}(\boldsymbol{z}; c)$ est celle de la VPP de MDS (§ 3), dont le foil a pour coefficients $\mu_k(\boldsymbol{z}, \boldsymbol{a})$ plus la correction FSI.

— **Le référentiel du gain :** la recherche part du profil actuel du C-foil, admissible une fois projeté sur les 10 modes. Le gain est donc mesuré par rapport au foil actuel.

Dans le code

`scripts/run_real_vpp.py` : `CONDITIONS`, puis `VPPSectionOptimizer.update` : la vitesse d'une section est `np.nanmean(speeds)` sur les 5 états (une condition non résolue est ignorée).

3 La VPP : MDS, Intens SY, variante `CFoilFSI`

3.1 Le système

La VPP est celle de MDS, branche `new_master`, avec le bateau `build_IntensSY(foil='CFoilFSI')` (C-foil de rayon 9,8 m). Elle reproduit vpp36 à $2 \cdot 10^{-6}$ nd près. C'est un `IntensSYSystem` de 9 sous-systèmes, évalués dans cet ordre (lu sur le système construit) :

Sous-système	Entrées (variables globales)	Modèle
`Weight`	masse, $X/Y/Z_{CG}$, gîte, assiette, dérive	poids au CG (pas de RS)
`Hull Dyn`	gîte, assiette, dérive, `CC_Dspl`, vitesse	RS de Chebyshev (efforts hydrodynamiques de coque)
`Hull Archi`	gîte, assiette, dérive, `CC_Dspl`	RS (hydrostatique, et l'enfoncement `CC_Sink_Out`)
`Foil Dyn`	gîte, dérive, assiette, `Foil_Cant/Rake/Yaw`, vitesse	**RS du C-foil, remplacée par notre surrogate**
`Foil Archi`	idem sans la vitesse	RS (poussée d'Archimède du foil)
`Rudder`, `Rudder STBD`	gîte, assiette, dérive, angle de safran, vitesse, `Hvol`	RS des deux safrans
`Sail`	dérive, `Flat`, TWA, TWS, vitesse, gîte, assiette, quête du mât	RS des voiles (entrées AWA, `Flat`)
`Windage`	gîte, assiette, dérive, TWA, TWS, vitesse	RS du fardage

Les équations d'équilibre. `TSystem.t_eval` évalue chaque sous-système. Chacun rend son torseur (forces et moments) transporté au repère de référence `CG ref`, situé en $(X_{CG}, 0, 0)$. La somme donne les 6 résidus `Eq.Fx` … `Eq.Mz`. Avant cela, `IntensSYSystem.update_system_frame` fait deux opérations :

— il ferme la hauteur d'assise `Hvol`, à partir de l'enfoncement statique et dynamique de la coque ;

— il tourne les bras de levier (les positions des repères des composants) selon l'attitude du bateau. Les forces, elles, sont déjà dans le repère monde.

3.2 Le problème résolu à chaque condition

Les 7 inconnues $\boldsymbol{u}$ et leurs bornes, pour la RS de coque `lin85t` :

Inconnue	Sens	Bornes	Départ
`Speed`	vitesse du bateau [nd]	$[6, 24]$	12
`Leeway`	dérive [°]	$[-6, 6]$	3
`Heel`	gîte [°]	$[0, 8]$	3
`Trim`	assiette [°]	$[-1{,}5;\ 1{,}5]$	0
`CC_Dspl`	déplacement porté par la carène [kg]	$[8500, 13500]$	11 540
`Rudder_Angle`	angle de safran [°]	$[-12, 12]$	0
`Flat`	réduction de puissance des voiles [–]	$[0{,}3;\ 1]$	0,6

$$V^\star(\boldsymbol{z};c) = \max_{\boldsymbol{u}} \; u_1 \quad \text{s.c.} \quad \frac{R_i(\boldsymbol{u};\boldsymbol{z},c)}{e_i} = 0,\; i = 1..6, \qquad \boldsymbol{u}_{\min} \le \boldsymbol{u} \le \boldsymbol{u}_{\max},$$

avec R_i les résidus `Eq.*` et $e = (10^3, 10^3, 10^3, 10^4, 10^4, 10^4)$ (N et N · m mis à la même échelle).

— **Le solveur :** SLSQP de SciPy (`ftol` 10^{-9}, 400 itérations au plus).
— **Le succès :** il faut que SLSQP converge *et* que $\|R/e\| < 10^{-4}$.
— **La relance :** en cas d'échec, une seule. Elle part de l'état initial fixe si un départ avait été donné, sinon de ce départ avec 2 nd de plus et 2° de gîte en moins.
— **Les commandes du foil :** `Foil_Cant`, `Foil_Rake` et `Foil_Yaw` restent à 0, comme dans les cas de navigation vpp36. Il n'y a pas de commande de foil.

Subtilité — dépendance au chemin

Le solve n'est pas une fonction pure de ses entrées : MDS garde un état interne d'un solve au suivant, par exemple la fermeture de l'enfoncement. Le 21/10, la même section et le même départ ont donné 14,83 nd une fois et 17,58 nd après d'autres solves. D'où trois règles :
— à chaque mise à jour, les sections sont résolues en partant des états du titulaire (départ « chaud ») ;
— des départs supplémentaires sont possibles (`extra_guesses`), et l'on garde la vitesse convergée la plus haute ;
— une différence finie compare deux solves **du même processus** (§ 3.5).

Subtilité — pas d'extrapolation en attitude

Les commandes du foil sont nulles, donc $\boldsymbol{a}$ = (gîte, assiette, dérive). Les bornes de ces trois inconnues, $[0, 8] \times [-1{,}5; 1{,}5] \times [-6, 6]$, sont exactement la boîte de la RS du foil, et aussi celle du plan d'expériences du surrogate. Un état VPP ne sort donc jamais du domaine du surrogate en attitude.

3.3 Le foil dans la VPP

Le composant `IntensSY_foil_dyn`, dans `intenssy_subsystems.py` (dossier `Systems/Ships/Sail/IntensSY` de MDS), fait quatre choses :

1. il calcule l'attitude effective :
$$a_c = \text{gîte} + \text{Foil_Cant}, \qquad a_r = \text{assiette} + \text{Foil_Rake}, \qquad a_y = \text{dérive} + \text{Foil_Yaw};$$
2. il assemble le vecteur d'entrée de la RS **par nom**, d'après ses `input_names` (`foil_rs_vector`). La vitesse n'est passée que si la RS déclare l'entrée `Speed_kts` ;
3. il appelle `regression(x)`, puis lit `prediction` et `out('Cpmin')` ;
4. il redimensionne les 6 premières sorties par $qS = \frac{1}{2}\rho_{\text{eau}} S V^2$, avec $S = 2{,}275$ m² et V en m/s. Le torseur est appliqué à l'origine du repère du foil, le bord d'attaque de la cale basse : (10,5745; 3,69217; −0,606084) m dans le repère du bateau.

Bending et **Twisting**, quand la RS les a, sont des sorties de rapport. Elles ne sont pas multipliées par qS et n'entrent pas dans l'équilibre.

Le remplacement par le surrogate. `SurrogateFoilRS` (`src/vpp_coupling.py`) a les attributs que MDS lit sur une `VppChebyRS` : `input_names`, `output_names`, `prediction`, `eval`, `regression`, `out`, `data`, les drapeaux d'extrapolation. `IntensSYVPP.set_foil` le met à la place de la RS du composant `Foil Dyn`. À chaque appel de MDS, avec $x = (a_c, a_r, a_y[, V])$:

$$\text{prediction} = \big(\mu_1..\mu_7\big)(\boldsymbol{z},\boldsymbol{a}) \; + \; \underbrace{r(\boldsymbol{z})\, b_{1..7}(\boldsymbol{a})\, V^2}_{\text{correction FSI}}, \qquad (\text{Bending}, \text{Twisting}) = r(\boldsymbol{z})\, b_{8,9}(\boldsymbol{a})\, V^2.$$

La section $\boldsymbol{z}$ est fixée pour un solve.

Ce qui n'est pas fait

— **Pas d'entrée Hvol :** le foil ne voit pas la hauteur d'assise. C'est un choix du 22/10, et la RS de production non plus ne la voit pas.
— **La VPP utilise la moyenne du surrogate**, et non sa loi : la variance ne se propage pas dans l'équilibre.
— **Cpmin n'entre ni dans l'objectif ni dans une contrainte.** Il est modélisé et rapporté seulement.

3.4 La correction FSI (foil flexible)

La RS FSI de production (`CFoil_R9800_S3250_C700_fsi_rs_vpp.json`) vient de bdFoil `CFoil_fsi.ipynb` : AVL × XFOIL avec un modèle de poutre « supported », sur 625 points, avec la vitesse $V \in [7, 28]$ nd en 4e entrée. Elle donne l'effet de la déformée pour le profil actuel.

La déformée suit la charge, donc V^2 (structure linéaire), et son effet sur les coefficients aussi : $C_k(\boldsymbol{a}, V) = C_k^{\text{rigide}}(\boldsymbol{a}) + b_k(\boldsymbol{a})V^2$. On l'a vérifié sur la RS : l'écart à 7 nd divisé par $V^2 - 49$ est constant à 3 % près jusqu'à 24 nd, et la loi suit la RS à 1,3 % près au pire sur les forces. La pente vient d'une régression sur $V_j \in \{7; 10{,}5; 14; 17{,}5; 21; 24{,}5; 28\}$:

$$b_k(\boldsymbol{a}) = \frac{\sum_j \left(V_j^2 - \overline{V^2}\right)\left(C_k^{RS}(\boldsymbol{a}, V_j) - \overline{C_k^{RS}}\right)}{\sum_j \left(V_j^2 - \overline{V^2}\right)^2}, \qquad \delta_k(\boldsymbol{z}, \boldsymbol{a}, V) = r(\boldsymbol{z})\, b_k(\boldsymbol{a})\, V^2, \qquad r(\boldsymbol{z}) = \frac{I_{xx}(\text{profil actuel})}{I_{xx}(\mathcal{D}(\boldsymbol{z}))}.$$

— **Le domaine :** l'attitude est ramenée dans la boîte de la RS. La loi en V^2 vaut aussi sous 7 nd, sans extrapoler la RS.

— **Le rapport r :** un profil plus raide se déforme moins. La flexion va en $1/EI$, la torsion en $1/GJ$, et les deux sont prises proportionnelles à I_{xx} (même famille, même corde, même structure). Avec la contrainte $I_{xx} \geq I_{xx}^{\text{actuel}}$, on a $r \leq 1$.

— **Le contrôle :** la VPP avec « partie rigide de la RS + δ » redonne la VPP de production à 0,0005 nd près à 45/16, et à 0,007 nd près à 90/14 (test `test_fsi_correction_reproduces_the_production_vpp`).

— **L'effet :** la déformée fait gagner 0,075 nd à 45/16 et 0,31 nd à 90/14.

Subtilité — hypothèses de la correction

Elle suppose deux choses :

— à attitude égale, la section porte les charges du profil actuel, à quelques pour cent près ;

— la structure est celle de la campagne de production.

C'est une correction de premier ordre. Un calcul FSI des profils comparés (bdFoil `simulation_fsi`) la vérifierait.

3.5 Les sensibilités de vitesse

$d_{kc} = \partial V^\star / \partial C_k$ se calcule par différences finies avant, à l'état du titulaire et pour chaque condition : on décale la sortie k du surrogate de h_k, puis on refait le solve depuis l'état de base :

$$d_{kc} \approx \frac{V^\star(\mu + h_k \boldsymbol{e}_k) - V^\star(\mu)}{h_k}, \qquad h = (5 \cdot 10^{-4},\ 10^{-2},\ 10^{-2},\ 2 \cdot 10^{-2},\ 3 \cdot 10^{-3},\ 3 \cdot 10^{-3})$$

pour $F_x, F_y, F_z, M_x, M_y, M_z$, soit environ 3 % de chaque coefficient. Le décalage s'applique avant la correction FSI.

Dans le code — `_vpp_task`, branche « sensitivity »

Chaque tâche refait d'abord le solve de base *dans son propre processus*, puis le solve décalé. Avec un état de base calculé par un autre processus, le premier test court donnait $\partial V/\partial F_x = -32$ au lieu de $+83$, à cause de la dépendance au chemin.

— Une dérivée non calculable (NaN) reprend sa valeur précédente, ou 0 la première fois.

— Les sensibilités sont recalculées une mise à jour sur trois.

Au 3e run, d_{F_x} allait de 84 (45/12) à 302 (90/14) nd par unité de F_x. Les autres dérivées sont plus petites d'un à deux ordres.

4 Les codes du torseur

Le foil est celui de la RS de production (bdFoil `CFoil_generation.ipynb`) :

— un arc circulaire `cfoil_arc` de rayon 9,8 m et de 3,25 m d'envergure ;

— une corde de 0,70 m constante (effilement 1, vrillage nul), 41 stations ;

— une tangente d'emplanture à `cant_geom` = 20,168°, l'extrémité vers l'intérieur (`tip_side` $= -1$) ;

— une image de paroi en $z = 0$ (AVL `iZsym` $= +1$; NPLLT `surface_type` 0).

Le repère et l'attitude. Le repère bdFoil a son origine au bord d'attaque de la cale basse, x vers l'avant, y vers bâbord, z vers le haut. Les moments sont réduits à cette origine, comme dans la RS. Les entrées passent telles quelles : cant $= a_c$, rake $= a_r$, yaw $= a_y$, sink $= 0$.

L'écoulement. $Re = 5 \cdot 10^6$ sur toute l'envergure, $n_{\text{crit}} = 1$, transition à 10 % sur les deux faces, table d'incidence de $-6°$ à $12°$ par pas de $0{,}5°$.

Le profil. 121 points par face (AVL ne lit pas 161 points et échoue sans rien écrire), avec un bord de fuite d'épaisseur $0{,}00197\,c$.

Niveau	Modèle	Cpmin
1 (BF)	ligne portante non plane (Phillips & Snyder, `bdFoil/NonPlanarSolver`), 100 nœuds, rayon de cœur 0,005 corde, polaires NeuralFoil xxxlarge. Temps : 0,2 à 0,4 s.	20e percentile, sur les points de contrôle, du Cpmin NeuralFoil à l'incidence locale $(1-(u_e/V)^2$, stations de couche limite $x/c \geq 1{,}6$ %)
2 (HF)	la même ligne portante avec les polaires XFOIL (`npane` 250, `niter` 300, 120 s au plus). Temps : 2 à 13 s pour la polaire d'une nouvelle section, puis 0,1 s par attitude.	20e percentile du cpmin de la polaire XFOIL à l'incidence locale. La polaire est relue dans le cache.
3 (validation)	AVL (bdFoil core) : traînée induite au plan de Trefftz, traînée de profil = CD de la polaire XFOIL au cl de chaque bande (modèle « profile »). Temps : 4,8 s par appel.	20e percentile des Cpmin de bande (`cp_min_perc` 20), comme la RS

— **Les sorties.** Les sorties 1 à 6 sont les coefficients bruts de bdFoil, `Cx` … `Cmz`, sans conversion. L'étape 0 a montré qu'ils retrouvent la RS de production : pentes de 0,86 à 0,96 et $R^2 \geq 0{,}996$ pour AVL.

— **Les échecs.** Un calcul raté (NPLLT non convergé, polaire XFOIL ratée, décrochage hors de la zone d'emplanture de 2 % d'envergure) rend des sorties NaN avec son message d'erreur.

— **Les deux niveaux du surrogate.** Sur le profil actuel, NeuralFoil xxxlarge et XFOIL donnent les mêmes forces à 0,1 % près. Ils ne diffèrent vraiment que sur Cpmin : NeuralFoil ne voit pas le pic de bord d'attaque.

Dans le code

`src/foil_codes.py` : `PLANFORM`, `FLOW`, `LEVELS_XFOIL`, et `FoilCodes.evaluate(weights, attitude, level)`, qui rend `(y, info)`. Les backends sont dans `pipelines/bdtoolbox_foil/solvers.py` (`NpLltBackend`, `AvlCoreBackend`).

— Le cache des polaires XFOIL a pour clé le sha256 du fichier `.xf` et les réglages. Il publie par un seul `os.replace`, donc deux processus qui calculent la même section ne se gênent pas : le premier qui écrit gagne.

— Chaque processus écrit sa propre configuration `codes_config_<pid>.json`.

5 La section : Kulfan, famille admissible, PCA

Kulfan (CST). Le poids de bord d'attaque est nul et l'épaisseur de bord de fuite TE fixée. Avec 8 poids par face :

$$y_{\text{ext}}(x) = \sqrt{x}(1-x)\sum_{i=0}^{7} u_i B_i(x) + x\,\tfrac{TE}{2}, \qquad y_{\text{int}}(x) = \sqrt{x}(1-x)\sum_{i=0}^{7} l_i B_i(x) - x\,\tfrac{TE}{2},$$

où B_i sont les polynômes de Bernstein de degré 7. Aux stations fixées, les coordonnées sont une fonction affine des 16 poids : $\boldsymbol{y} = A\boldsymbol{w} + \boldsymbol{b}$.

Le profil actuel. Ce sont les poids de référence de l'étude `cfoil_intens_kulfan_trim_2` (`CFoilFamily.from_study_config(...).reference_weights()`) : l'épaisseur du MC2 et la cambrure de référence ($\delta = 0{,}108139$), soit 12,25 % d'épaisseur et $I_{xx} = 6{,}7174 \cdot 10^{-5}$ (corde unité).

La famille $\mathcal{F}$ (2e et 3e runs). Une section appartient à la famille si elle remplit quatre conditions :

1. ses 16 poids sont dans une boîte centrée sur le profil actuel, de largeur la moitié de celle de la boîte de `Dim-Reduction/experiments.ipynb` (`KulfanBoxFamily.centered`, `notebook_bounds`) ;
2. ses faces ne se croisent pas : l'épaisseur locale est ≥ 0 sur 2001 stations ;
3. $I_{xx} \geq I_{xx}^{\text{actuel}}$, avec $I_{xx} = \int \left[\tau^3/12 + \tau(y_c - y_g)^2\right] dx$, τ l'épaisseur, y_c la ligne moyenne et y_g le centre de gravité (`section_ixx`) ;
4. NeuralFoil xxxlarge est confiant : la confiance d'analyse est $\geq 0{,}9$ à 0, 2, 4 et 6° (`confident`).

Les tirages se font par hypercube latin dans la boîte, puis on rejette les sections hors famille et on retire jusqu'à en avoir assez (`KulfanBoxFamily.sample`).

La PCA.

— **L'ajustement :** sur 4000 sections de la famille. On prend leurs coordonnées à 200 stations uniformes, on les centre (sans les standardiser), puis on fait une SVD : $Y - \bar{\boldsymbol{y}} = USV^{\mathsf{T}}$.

— **Les scores** sont blanchis : $z_j = (\boldsymbol{y} - \bar{\boldsymbol{y}}) \cdot \boldsymbol{v}_j / \sigma_j$.

— **Le décodage** est exact : chaque mode est $\boldsymbol{v}_j = A\boldsymbol{m}_j$, d'où $\mathcal{D}(\boldsymbol{z}) = \bar{\boldsymbol{w}} + \sum_{j \leq k} \sigma_j z_j \boldsymbol{m}_j$.

— **Le nombre de modes :** la famille a un rang de 16, et $k = 10$ modes portent 99,86 % de la variance géométrique. Ces 10 scores forment l'espace de recherche.

Subtilité — l'admissibilité se vérifie sur la section décodée

La troncature à 10 modes retire un peu d'I_{xx} : 24 % des sections projetées sortaient de la famille. `RealFoilProblem.sample_sections` tire donc des sections de la famille, les encode, puis **ne garde que les scores dont la section décodée est encore dans la famille**. Celle-ci est la section que voient les codes. Les bornes des scores (pour normaliser le surrogate) sont les minimums et maximums de 4096 tirages admissibles.

6 Le surrogate du torseur : 7 co-krigeages AR(1)

La structure. Il y a un co-krigeage `CoKriging` de mfego par sortie : 7 modèles indépendants (F_x … M_z, $C_{p,\min}$) sur le **même plan imbriqué**. Un calcul de code donne en effet tout le torseur (`TorsorCoKriging`, `src/vpp_targeted.py`). Les entrées $\boldsymbol{x} \in \mathbb{R}^{13}$ sont ramenées dans $[0,1]^{13}$ avec les bornes du problème.

Le modèle auto-régressif (Kennedy & O'Hagan), pour chaque sortie :

$$Z_1(\boldsymbol{x}) = \delta_1(\boldsymbol{x}), \qquad Z_2(\boldsymbol{x}) = \rho\, Z_1(\boldsymbol{x}) + \delta_2(\boldsymbol{x}),$$

où δ_1 et δ_2 sont des GP indépendants, chacun avec une moyenne constante et le noyau

$$k(\boldsymbol{x},\boldsymbol{x}') = t_1\Big(1+\sqrt{5}\,r+\tfrac{5}{3}r^2\Big)e^{-\sqrt{5}\,r} + t_2, \qquad r^2 = \sum_{m=1}^{13}\frac{(x_m-x'_m)^2}{\ell_m^2}$$

(Matérn 5/2 à longueurs de corrélation par dimension, plus une constante), avec un bruit de pépite.

Les hyperparamètres sont estimés niveau par niveau : sur un plan imbriqué sans bruit, la vraisemblance se factorise. On les estime par MAP (L-BFGS-B, 3 relances), avec les priors suivants (sorties normalisées, entrées dans $[0,1]$) :

Paramètre	Prior	Bornes
ℓ_m	InvGamma(3, 2), mode 0,5	$[10^{-2}, 10]$
t_1	LogNormale(1, 1)	$[10^{-4}, 10^{2}]$
t_2	LogNormale($\ln 10^{-2}$, 2)	$[10^{-8}, 10^{2}]$
bruit	LogNormale($\ln 10^{-4}$, 2)	$[10^{-8}, 10^{-2}]$
ρ	Normale(1, 2^2) (« v2 »), profilé en forme close, centré	$[-5, 5]$

Le niveau 2 s'ajuste sur le résidu $z_2 - \rho\, z_1(X_2)$, où $z_1(X_2)$ sont les valeurs *observées* du niveau 1 aux points HF. Le résidu est centré, donc sa moyenne est nulle quel que soit ρ. Pour Θ fixé, $\hat\rho$ est l'optimum en forme close sous le prior normal.

La prédiction conditionne $Z_2(\boldsymbol{x})$ sur *toutes* les observations $\boldsymbol{z}$ (des deux niveaux), avec la matrice de covariance jointe V, factorisée une fois par Cholesky ($V = LL^{\mathsf{T}}$) :

$$\mu(\boldsymbol{x}) = m_2 + \boldsymbol{k}(\boldsymbol{x})^{\mathsf{T}} V^{-1}(\boldsymbol{z}-\boldsymbol{m}), \qquad s^2(\boldsymbol{x}) = c_{22}(\boldsymbol{x},\boldsymbol{x}) - \boldsymbol{k}(\boldsymbol{x})^{\mathsf{T}} V^{-1}\boldsymbol{k}(\boldsymbol{x}),$$

avec $\mathrm{Cov}(Z_i(\boldsymbol{x}), Z_j(\boldsymbol{x}')) = \sum_{m\le\min(i,j)} P(m,i)P(m,j)\,\sigma^2_{y,m}\,k_m(\boldsymbol{x},\boldsymbol{x}')$ et $P(1,2)=\rho$.

Dans le code — ce qui coûte, et comment c'est réparti

- `fit(refit=True)` : de nouveaux hyperparamètres (MAP). `fit(refit=False)` : on garde les hyperparamètres, et l'on reconditionne seulement (Cholesky de V, $\alpha = V^{-1}(\boldsymbol{z}-\boldsymbol{m})$). Les 7 modèles tournent dans 7 fils (`TorsorCoKriging.map_models`), et le résultat est identique au calcul en série.
- **Le rythme des réajustements :** de nouveaux hyperparamètres dès que le plan a grossi de 25 % depuis le dernier ajustement (`refit_growth` = 1,25). Au 3e run, cela s'est produit à 314, 408, 518, 651, 830 et 1045 points. Sinon, on reconditionne seulement.
- **Pour la VPP, la moyenne seule :** `TorsorCoKriging.predict` appelle `latent_mean`, soit $m_2 + \boldsymbol{k}(\boldsymbol{x})^{\mathsf{T}}\alpha$, en $O(n)$. La variance demanderait une résolution triangulaire en $O(n^2)$ par point, et la VPP appelle le foil des centaines de fois par solve.
- **Les workers VPP** reçoivent une copie sans le facteur de Cholesky (`mean_only_copy`), qui ne sait prédire que les moyennes.

7 Le critère de raffinement ciblé par la VPP

L'idée. À un état d'équilibre, une erreur ΔC_k sur le coefficient k change la vitesse de $d_k \Delta C_k$ environ. Les modèles sont indépendants, donc au premier ordre

$$\mathrm{Var}\, V \approx \sum_{k=1}^{6} d_k^2\, s_k^2(\boldsymbol{x}).$$

On cherche à réduire cette variance **en moyenne sur les points d'intégration**, qui entourent les états VPP des sections en compétition. Cpmin a un poids nul : il n'agit pas sur la vitesse.

Le poids des sorties. $w_k = \frac{1}{5}\sum_c d_{kc}^2$ (moyenne sur les conditions des sensibilités du titulaire).

Les points d'intégration $\{\boldsymbol{x}_i^{\text{int}}\}_{i=1}^N$. À chaque mise à jour, on prend les 20 états (4 sections × 5 conditions). Autour de chacun, on tire un nuage de Sobol de 16 points, où la section reste fixée et l'attitude varie de $\pm(1°; 0{,}3°; 1°)$ en (cant, rake, yaw), dans la boîte. Cela fait $N = 320$ points.

Le gain exact d'une observation (ALC/IMSE, `batched_imse_gains`). Soit un candidat $\boldsymbol{x}_m$ et un niveau l. Le plan doit rester imbriqué : on ajoute les observations $(\boldsymbol{x}_m, j)$ pour tous les niveaux $j \le l$ qui manquent en $\boldsymbol{x}_m$. Avec $\boldsymbol{c}_i$ la covariance a posteriori entre $Z_2(\boldsymbol{x}_i^{\text{int}})$ et ces observations, et S leur covariance a posteriori (bruit compris), la réduction exacte de la variance de Z_2 en $\boldsymbol{x}_i^{\text{int}}$ est $\boldsymbol{c}_i^{\mathsf{T}} S^{-1} \boldsymbol{c}_i$. D'où

$$G_k(\boldsymbol{x}_m, l) = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{c}_i^{\mathsf{T}} S^{-1} \boldsymbol{c}_i.$$

Le calcul se fait pour tous les candidats d'un coup (tableaux à un axe de plus), avec les blocs $W = L^{-1}C(\mathcal{D}, \cdot)$.

Le rendement et le choix.

$$(\boldsymbol{x}^\star, l^\star) = \arg\max_{m,\,l} \frac{\sum_{k\le 6} w_k\, G_k(\boldsymbol{x}_m, l)}{C(\boldsymbol{x}_m, l)}, \qquad C(\boldsymbol{x}_m, l) = \sum_{j\le l,\ j \text{ manquant}} c_j, \qquad c = (1, 2).$$

Pour un point nouveau, le niveau 2 coûte $1 + 2 = 3$. Pour un point BF sans HF, il coûte 2.

Subtilité — le rapport de coût 1 : 2

Les codes d'un lot tournent en même temps. Un choix coûte surtout le temps de sélection, de 1 à 6 s, à peu près le même aux deux niveaux. Un calcul HF n'allonge le lot que si sa section est nouvelle (la polaire XFOIL). Au 2e run, en séquentiel avec AVL, le rapport valait 1 : 6.

Les candidats (`VPPSectionOptimizer.candidates`).

— 500 points des nuages autour des états (25 par état).
— 256 points de tout le domaine : des sections admissibles nouvelles, avec une attitude de Sobol dans la boîte.
— Au plus 512 points BF sans HF, les plus récents, pour lesquels seul le niveau 2 est possible.
— On retire les candidats à moins de 10^{-3} (cube unité, norme infinie) d'un calcul raté.

8 Les lots en parallèle : le « kriging believer »

On veut 16 choix par lot pour faire tourner les codes en parallèle. Ce n'est pas un choix des 16 meilleurs rendements d'un coup : ces points seraient presque tous voisins et redondants. On choisit plutôt l'un après l'autre. Après chaque choix, on conditionne le co-krigeage sur le point choisi comme s'il avait été calculé, avec pour valeur provisoire la **moyenne prédite du niveau concerné**.

Pourquoi c'est exact pour notre critère. La covariance a posteriori d'un GP,

$$k_n(\boldsymbol{a}, \boldsymbol{b}) = k(\boldsymbol{a}, \boldsymbol{b}) - \boldsymbol{k}(\boldsymbol{a}, X) K^{-1} \boldsymbol{k}(X, \boldsymbol{b}),$$

ne dépend que des *positions* des observations, et non de leurs valeurs. Le gain G_k du choix suivant voit donc exactement la réduction de variance apportée par les choix précédents. La valeur provisoire laisse les moyennes inchangées (test `test_kriging_believer_placeholder`). Le lot s'étale tout seul.

9 L'optimiseur de section

Il y a une mise à jour avant chaque lot (`update_every` = 1, `VPPSectionOptimizer.update`).

1. **La publication** du surrogate courant pour les workers VPP.
2. **Les sensibilités** d_{kc}, une mise à jour sur trois.
3. **Le criblage** (`screen`). On tire 256 sections admissibles nouvelles, avec la graine seed + 31 $n_{\text{mises à jour}}$, et on y ajoute le titulaire. On garde fixe l'attitude $\boldsymbol{a}_c$ des états du titulaire et sa vitesse V_c. Le changement de vitesse au premier ordre vaut alors

$$\Delta V_c(\boldsymbol{z}) = \sum_{k\le 6} d_{kc}\big[\mu_k(\boldsymbol{z}, \boldsymbol{a}_c) - \mu_k(\boldsymbol{z}_{\text{tit}}, \boldsymbol{a}_c)\big] + \big(r(\boldsymbol{z}) - r(\boldsymbol{z}_{\text{tit}})\big) \sum_{k\le 6} d_{kc}\, b_k(\boldsymbol{a}_c) V_c^2,$$

où le second terme est la différence des corrections FSI. On calcule aussi

$$\sigma(\boldsymbol{z}) = \tfrac{1}{5}\Big(\sum_c \sum_{k\le 6} d_{kc}^2\, s_k^2(\boldsymbol{z}, \boldsymbol{a}_c)\Big)^{1/2}, \qquad \text{score}(\boldsymbol{z}) = \tfrac{1}{5}\sum_c \Delta V_c(\boldsymbol{z}) + \kappa\, \sigma(\boldsymbol{z}), \quad \kappa = 1.$$

Algorithme 1 Un lot (`VPPSectionOptimizer.step`)

$n \leftarrow$ taille du plan ; **si** $n \geq 1{,}25\, n_{\text{dernier ajustement}}$ **alors** nouveaux hyperparamètres **sinon** reconditionnement
mise à jour de l'optimiseur (§ 9) : titulaire, états, points d'intégration, d_{kc}
$w_k \leftarrow \frac{1}{5}\sum_c d_{kc}^2$; candidats $\{\boldsymbol{x}_m\}$ et niveaux manquants
pour $b = 1..16$ **faire**
    $(\boldsymbol{x}_b, l_b) \leftarrow$ meilleur rendement (§ 7) ; marquer ses niveaux $\leq l_b$ comme faits
    si $b < 16$ **alors**
        pour chaque niveau $j \leq l_b$ manquant en $\boldsymbol{x}_b$ **faire** ajouter $(\boldsymbol{x}_b, j)$ avec la valeur $\mu^{(j)}(\boldsymbol{x}_b)$
        fin pour
        reconditionner les 7 modèles (hyperparamètres fixes)
    fin si
fin pour
retirer les valeurs provisoires (`truncate`)
lancer en parallèle tous les calculs (point, niveau) du lot (28 processus)
pour chaque choix b **faire**
    ajouter ses résultats, niveau 1 d'abord. Un résultat NaN, ou un HF dont le BF manque, est rejeté, et le point rejoint la liste des échecs
fin pour

Le terme $\kappa\sigma$ est un optimisme face à l'incertitude : une section dont la vitesse est incertaine mérite d'être regardée. On retient les 3 meilleurs scores.

4. **La VPP complète** sur ces 3 sections et sur le titulaire, aux 5 conditions, soit 20 solves en parallèle, avec départ aux états du titulaire. La section de meilleure vitesse moyenne devient le titulaire.
5. **Les points d'intégration** : nuages de Sobol autour des 20 états des 4 sections (§ 7).

Le départ et la fin.

— **Le départ :** le premier titulaire est le profil actuel projeté, et ses états sont résolus à la première mise à jour.
— **La boucle** s'arrête quand le temps écoulé depuis le début du run dépasse la limite. Le lot en cours se termine.
— **La fin :** un dernier ajustement des hyperparamètres, puis une dernière mise à jour. Celle-ci fixe le titulaire final avec le modèle final.

Subtilité — ce que la recherche explore vraiment

Le titulaire ne bouge que vers une section *tirée* au criblage : la recherche trie des réserves de sections aléatoires, environ 13 000 en 52 mises à jour au 3e run. Ce n'est pas un optimum local au sens strict.
Les réserves dépendent de la graine. Les 2e et 3e runs, de même graine, même famille et même PCA, ont donc criblé les mêmes sections. Ils ont choisi le même profil à la même mise à jour (la 14e), alors que leurs modèles différaient : AVL rigide d'un côté, NPLLT × XFOIL flexible de l'autre.

10 Le plan de départ

— **Le niveau 1 :** $n_1 = 256$ points, avec des sections admissibles tirées (§ 5) et une attitude par hypercube latin dans $[0, 8] \times [-1{,}5; 1{,}5] \times [-6, 6]$.
— **Le niveau 2 :** $n_2 = 64$ points choisis parmi les points BF réussis, par maximin glouton dans le cube unité (`greedy_maximin`). Le premier est le plus proche du centre, puis on ajoute à chaque fois le point le plus loin des points déjà pris. Le plan est donc imbriqué.
— **Les échecs** sont écartés. Au 3e run, il y a eu 253/256 BF et 61/64 HF réussis, en 1,4 min de calcul parallèle.

11 L'architecture logicielle et parallèle

Fichier	Rôle
`scripts/run_real_vpp.py`	orchestration en 5 étapes : famille et PCA, plan de départ, boucle, validation, résultats et figures
`src/section_pca.py`	Kulfan, `KulfanBoxFamily`, `SectionPCA`, `section_ixx`
`src/meta_foil.py`	lecteur de RS de Chebyshev (`ChebyshevRS`), filtre `confident`
`src/foil_codes.py`	`FoilCodes` : les 3 niveaux, Cpmin
`pipelines/bdtoolbox_foil/`	backends NPLLT, AVL et XFOIL (bdFoil)
`src/vpp_coupling.py`	`FSICorrection`, `SurrogateFoilRS`, `IntensSYVPP` (solve, sensibilités)
`mfego/src/cokriging.py`	`CoKriging` AR(1)
`src/vpp_targeted.py`	`TorsorCoKriging`, `latent_mean`, `choose`
`src/sequential_design.py`	`batched_imse_gains` (gain ALC exact)
`src/vpp_optimize.py`	`RealFoilProblem`, `CodeRunner`, `VPPPool`, `VPPSectionOptimizer`, `validate`
`src/run_dir.py`, `scripts/real_run_figures.py`	dossier de run, figures
`src/operating_profile.py`, `profiles/*.json`	distribution d'usage : mélange de modes, nœuds et poids
`scripts/profile_analysis.py`	comportement des sections sur une distribution d'usage (§ 13)

Les processus (fig. 1), sur un Ryzen 9 5950X de 16 cœurs (32 logiques) avec 128 Go de mémoire.

— **Le processus principal :** la boucle, le surrogate et le critère. Il a 2 fils BLAS, et les 7 modèles tournent en 7 fils pendant l'ajustement, le conditionnement, les variances et les gains.

— **`CodeRunner` : 28 processus.** Chacun a son `FoilCodes`. Le pool est créé au premier lot de plusieurs points, puis gardé pendant tout le run. Un point seul tourne dans le processus principal.

— **`VPPPool` : 30 processus.** Chacun a son `IntensSYVPP`, avec la correction FSI. À chaque mise à jour, le processus principal écrit `data/surrogate_v<n>.pkl`, la copie « moyennes seules ». Un worker ne relit le fichier que si la version a changé, et le fichier précédent est supprimé.

— **Les workers des deux pools** démarrent avec un seul fil BLAS (variables d'environnement posées avant leur création). Les deux pools ne travaillent jamais en même temps.

Le dossier d'un run, `runs/<date>_<nom>/` :

— `config.json` : tous les réglages ;

— `run.log` ;

— `data/evaluations.csv` : un calcul de code par ligne, avec le niveau, les entrées, les sorties, le temps et l'erreur ; aussi `data/pca.json` et `data/codes/` (configurations, cache des polaires) ;

— `results/` : `history.csv` (un choix par ligne), `updates.json`, `validation.csv`, `best_section.json`, `summary.txt`, `surrogate.pkl` (le surrogate final, copie « moyennes seules »), et `profile_<nom>.csv|json` si une distribution d'usage est analysée ;

— `figures/` : convergence, points, sections, validation, vitesses, et `profile_<nom>`.

12 La validation finale

Pour chaque section validée (le meilleur profil, le profil de départ et les meilleurs profils des runs antérieurs, donnés par `--compare`) :

1. on résout la VPP aux 5 conditions avec le surrogate final et la FSI ;
2. on lance les codes de niveaux 1, 2 et 3 aux 5 états, en parallèle ;
3. on corrige la vitesse au premier ordre de l'écart mesuré entre le surrogate et le code :

$$V_{\text{code}} \approx V_{\text{surrogate}} - \sum_{k \le 6} d_{kc}\big(\mu_k - C_k^{\text{code}}\big).$$

Le code est soit le niveau 2, soit AVL. On compare aussi l'erreur à l'écart type annoncé, $\big(\sum_k d_{kc}^2 s_k^2\big)^{1/2}$.

On n'a pas résolu la VPP avec un code dans la boucle : avec AVL, cela prendrait environ 30 min par condition.

Le résultat du 3e run.

— **Le gain** du meilleur profil sur le profil actuel vaut +0,241 nd en moyenne vu par NPLLT × XFOIL, et +0,217 nd vu par AVL × XFOIL. Il tient aux 5 conditions, de +0,02 à +0,40 nd.

— **Le Cpmin** passe de −2,57 à −0,87.

— **La précision :** l'erreur du surrogate contre NPLLT × XFOIL vaut 0,010 nd RMS au meilleur profil.

13 L'analyse sur une distribution d'usage

Le principe. La recherche optimise la vitesse moyenne sur les conditions du run, et le co-krigeage reste celui des sections précédentes (décision du 22/10). Le bateau, lui, navigue selon une distribution de (TWA, TWS), en général bimodale : un mode au près, un mode au portant. On ne la connaît pas encore. Elle se donne donc **à chaque run**, dans un fichier JSON, et elle sert à **analyser** les sections retenues : le meilleur profil, le profil de départ et les profils comparés.

La distribution (`src/operating_profile.py`) est un mélange de modes. Chaque mode m a un poids π_m, une loi normale tronquée de TWA et une loi normale tronquée de TWS, indépendantes dans le mode :

$$p(\mathrm{TWA},\mathrm{TWS}) = \sum_m \pi_m \,\mathcal{N}_{[a_m,b_m]}(\mathrm{TWA};\bar\theta_m,\sigma_{\theta,m})\,\mathcal{N}_{[c_m,d_m]}(\mathrm{TWS};\bar v_m,\sigma_{v,m}).$$

— **La quadrature.** Elle est produit par mode : n nœuds par variable, aux points de probabilité médiane de n strates équiprobables (les quantiles $(i+\frac12)/n$ de la loi tronquée), chacun de poids π_m/n^2. On a donc $\mathbb{E}_p[Q] \approx \sum_{\text{nœuds}} w\,Q$.

— **Les bornes.** Les nœuds restent dans les bornes : pas de queue hors du domaine de la VPP.

— **Des nœuds explicites.** Un fichier peut aussi donner ses nœuds directement, par exemple un histogramme de conditions enregistrées. Les poids sont alors normalisés à 1.

L'hypothèse actuelle (`profiles/guess_bimodal.json`, à remplacer) :

— un mode « près », TWA $45 \pm 4°$ sur $[38, 52]$;

— un mode « portant », TWA $135 \pm 7°$ sur $[120, 148]$;

— le même poids pour les deux, TWS 13 ± 3 nd sur $[8, 18]$ dans les deux modes ;

— 3×3 nœuds par mode, soit 18 nœuds.

L'analyse (`scripts/profile_analysis.py`, ou l'option `--profile` de `run_real_vpp.py`) se fait en quatre étapes :

1. on recharge le surrogate final du run ; un reconditionnement, sans changer les hyperparamètres, lui rend ses variances ;
2. on résout la VPP (avec la FSI si le run l'avait) à chaque nœud, pour chaque section, dans le pool VPP ;
3. on calcule à chaque nœud les sensibilités, puis l'écart type de la vitesse prédite $\left(\sum_k d_k^2 s_k^2\right)^{1/2}$. Le surrogate a été raffiné autour des états des conditions du run, et un nœud loin d'eux (au portant) est moins sûr ;
4. on prend les espérances **sur les nœuds résolus par toutes les sections**, pour comparer les mêmes conditions : $\mathbb{E}_p[V]$, la vitesse équivalente en temps $1/\mathbb{E}_p[1/V]$, l'écart type, les mêmes par mode, et le gain sur le profil de départ. La probabilité résolue par chaque section est donnée à part.

Subtilité — des nœuds sans équilibre

La VPP de l'Intens SY n'a pas d'équilibre au portant profond par petit temps. La gîte bute sur sa borne de 0°, avec un résidu non nul, et c'est vrai aussi avec la RS de production. Elle n'en a pas non plus par vent fort (45/20, 90/20 : vitesse à sa borne de 24 nd ou `Flat` au minimum). Ces nœuds sont signalés, et non cachés. Ils relèvent du réglage de la VPP (bornes, configuration de foil au portant), pas du surrogate.

Le résultat sur le 3e run. L'analyse s'est faite après coup, sur le surrogate `data/surrogate_v52.pkl`.

— **Les nœuds résolus :** les deux profils ensemble en résolvent 72 % de la probabilité, tout le près et 22 % sur les 50 % du portant.

— **Le gain** du meilleur profil sur le profil actuel vaut **+0,099 nd** : +0,085 nd au près et +0,13 nd au portant. Sur les 5 conditions du run, centrées sur le reaching, il valait +0,24 nd.

— **L'écart type** du surrogate est d'environ 0,006 nd. C'est l'estimation du modèle lui-même : le portant n'a pas été validé contre les codes.

14 Les réglages du 3e run

Réglage	Valeur
commande	`run_real_vpp.py --box current --box-scale 0.5 --hf npllt_xfoil --hf-cost 2 --fsi --batch 16 --workers 28 --vpp-workers 30 --n-low 256 --n-high 64 --time-limit-min 65 --compare <best_section.json du 2e run>`
PCA	$k = 10$, ajustée sur 4000 sections, graine 0
plan de départ	256 BF, 64 HF
lots	16 choix, rapport de coût 1 : 2
mise à jour	à chaque lot ; sensibilités une fois sur trois ; criblage de 256 sections, 3 retenues, $\kappa = 1$
points d'intégration	16 par état, demi-largeurs (1°; 0,3°; 1°)
candidats	25 par état (500), 256 dans tout le domaine, ≤ 512 BF sans HF
réajustement	croissance de 25 % du plan
co-krigeage	Matérn 5/2 à longueurs par dimension + constante, MAP, $\rho \sim \mathcal{N}(1, 2^2)$ sur $[-5, 5]$, 3 relances
FSI	RS `CFoil_R9800_S3250_C700_fsi_rs_vpp.json`, rapport I_{xx}
arrêt	65 min depuis le début du run (fin réelle : 69,7 min)

15 Limites connues et évolutions prévues

1. **La recherche par tirages.** Il faudrait un raffinement local autour du titulaire (des candidats dans un voisinage qui se resserre, ou un gradient sur le surrogate), et un run avec une autre graine des réserves.
2. **La distribution d'usage** sert aujourd'hui à l'analyse seulement (§ 13). Si un jour on voulait chercher directement sur elle, quatre endroits du code changeraient :
   — la vitesse d'une section (`update`), qui deviendrait une moyenne pondérée par les poids ω_c des nœuds ;
   — le gain du criblage (`screen`) ;
   — les sensibilités, à calculer à chaque nœud ;
   — la moyenne sur les points d'intégration de `batched_imse_gains`, à pondérer par $\omega_{c(i)}\, d^2_{k\,c(i)}$.

   Une analyse qui montre un gain très différent selon le mode, comme au 3e run (+0,085 nd au près contre +0,13 nd au portant), est un signe que cela vaudrait la peine. Il faudrait aussi donner aux conditions du run les poids de la distribution, et rendre la VPP résoluble sur tout son domaine.
3. **Le coût de la sélection.** Le choix d'un lot, en $O(n^2)$, est devenu le goulot : de 25 à 100 s par lot de 16 au 3e run, pour un plan de 300 à 1300 points.
4. **Les sensibilités ponctuelles aberrantes** : une fois au 3e run, d_{F_x} a valu 201 au lieu de 111 pendant trois mises à jour. On pourrait prendre la médiane de deux solves, ou rejeter une valeur trop loin de la précédente.
5. **La FSI de premier ordre** et **la validation entre modèles potentiels** : quelques calculs CFD aux états VPP des profils comparés trancheraient.